

Disaggregated Serving Deep Dive

Author: Harshavardhana Srinivasan

Project: InferTutor Arena LLM inference ablation study.

Purpose

This document explains the attempted disaggregated prefill/decode serving prototype built during the InferTutor Arena ablation study. The goal was to evaluate whether separating prompt prefill from token decode could improve mixed multimodal serving under load.

The main experiment report records the result at a high level. This companion note explains the architecture, implementation path, observed failure, and what would be required to continue the direction in a production-grade follow-up.

Why Disaggregate Prefill and Decode

Transformer inference has two major execution phases:

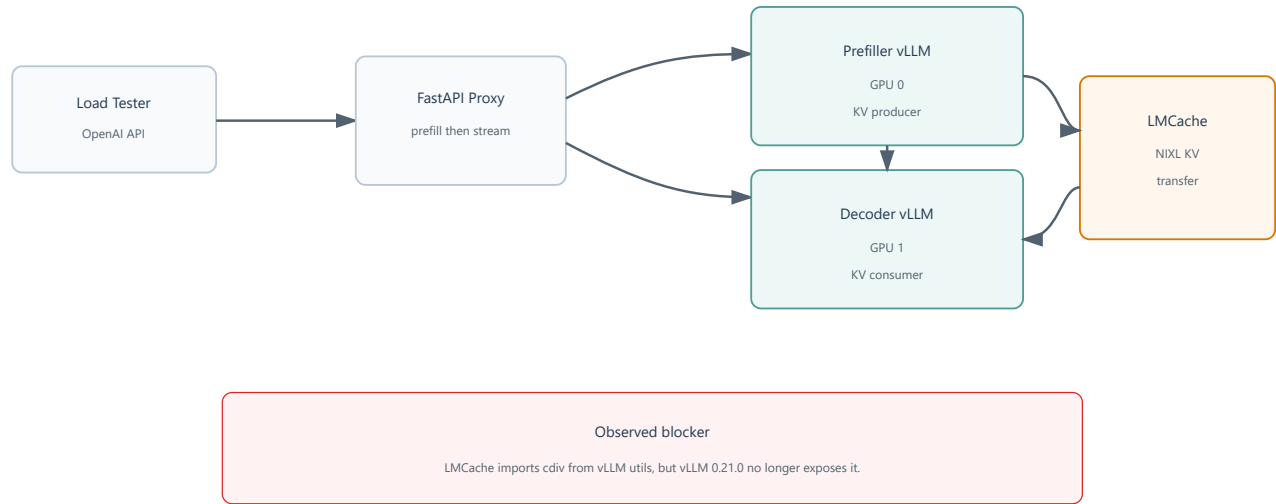
- **Prefill:** The model processes the full prompt and builds the KV cache. This phase is compute-heavy and can be especially expensive for long-context and image-conditioned requests.
- **Decode:** The model generates output tokens incrementally. This phase is latency-sensitive and usually benefits from steady scheduling and fast KV-cache access.

In a monolithic vLLM server, prefill and decode share the same scheduler and GPU worker. This is simpler and usually robust, but under mixed multimodal traffic, long or image-heavy prefill work can interfere with short decode work. The disaggregated serving hypothesis was that separating these phases could reduce interference.

Proposed Architecture

Attempted Disaggregated Prefill/Decode Architecture

A prefiller/decoder pair was attempted with LMCache KV transfer, but engine initialization failed.



Attempted disaggregated prefill/decode architecture

The prototype used three logical components:

Component	Responsibility
FastAPI proxy	Accept OpenAI-compatible requests and coordinate prefill/decode flow
Prefiller vLLM server	Process the prompt and produce KV-cache state
Decoder vLLM server	Consume KV-cache state and stream generated tokens
LMCache/NIXL layer	Transfer KV state between the prefiller and decoder

The intended request flow was:

1. A client submits a chat completion request to the proxy.
2. The proxy sends the prompt to the prefiller endpoint.
3. The prefiller computes prompt states and exports KV cache through LMCache.
4. The proxy sends decode work to the decoder endpoint.
5. The decoder imports the KV cache and streams the final response back to the client.

Implementation Files

The disaggregated prototype is captured in the following files:

File	Purpose
starter_code/modal_disagg_infertutor_app.py	Modal app definition for prefiller, decoder, and proxy services
starter_code/run_disagg_infertutor_experiment.py	Experiment launcher for the disaggregated serving attempt
report_assets/architecture_disaggregated.svg	Architecture diagram used in the report

The implementation was deliberately separated from the primary monolithic serving code so that the stable best configuration remained reproducible.

Modal Deployment Shape

The prototype attempted to run independent service processes inside Modal-managed GPU containers:

- A prefiller service configured as a vLLM server.
- A decoder service configured as a vLLM server.
- A proxy service exposing a health route and OpenAI-compatible completion route.

The deployment was more complex than the standard monolithic experiment because health required both backend roles to initialize successfully. If either prefiller or decoder failed, the proxy correctly returned a service-unavailable health state.

Observed Failure

The prototype reached vLLM startup, but both prefiller and decoder failed during LMCache integration. The key blocker was a package compatibility issue:

```
lmcache.integration.vllm.vllm_v1_adapter imported cdiv from vllm.utils,  
but the installed vllm==0.21.0 package did not expose cdiv.
```

As a result, the health endpoint repeatedly returned 503 because the proxy could not connect to healthy prefiller and decoder backends.

Decision

Due to the runtime error, we did not proceed further with the disaggregated prototype.

This was not treated as a score-producing experiment. It was documented as an attempted architecture prototype because the failure mode was architectural and dependency-related rather than a normal benchmark regression.

What This Attempt Demonstrated

The attempt still produced useful engineering evidence:

- Disaggregated serving requires version-aligned vLLM and KV-transfer dependencies.
- Health checks must validate both control-plane readiness and backend connectivity.
- A proxy can make failure modes visible instead of silently routing traffic to broken backends.
- Advanced serving architectures introduce integration risk in addition to latency and throughput risk.

Future Work

A stronger follow-up would use a pinned compatibility matrix:

Layer	Follow-up Requirement
vLLM	Pin a version known to work with LMCache disaggregated serving
LMCache	Pin matching integration package and verify adapter imports before deployment
NIXL/KV transfer	Validate transport availability and configuration in a small single-request test
Health checks	Add separate prefiller, decoder, and proxy readiness probes
Load testing	Start with one-user smoke tests before mixed 120-user traffic

A future successful run should first prove the end-to-end prefill/decode path on a single prompt, then compare against the monolithic b2048/seq32/u120 baseline under the same scoring formula.

Conclusion

The disaggregated serving work was a valuable prototype attempt but not a completed optimization. The main result of the study therefore remains the tuned monolithic 4-replica mixed vLLM configuration. Disaggregated serving remains a promising research direction, but in this environment it required dependency alignment work before it could be evaluated fairly.